INTERNAL TECHNICAL REFERENCE

Database Documentation

Complete reference for the Quiz Panel database — every table, every column, how the app-owned schema and the legacy university database fit together, and how each part is actually used across the product.

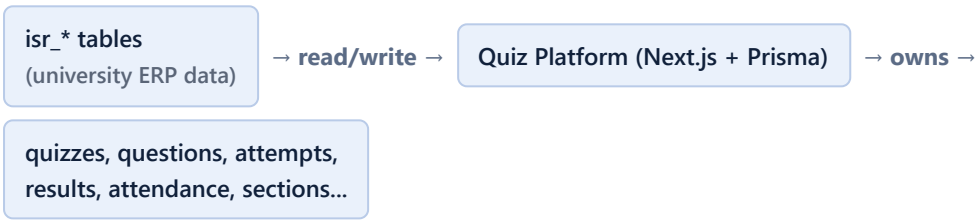
Generated 2026-08-17 · MySQL via Prisma ORM · 28 Prisma-managed tables + dynamic legacy registration tables

Contents

1. Architecture at a Glance

The application runs on a single MySQL database, but that database really holds **two schemas with different owners**:

- **App-owned tables** `APP` — everything the quiz platform itself invented: quizzes, questions, attempts, answers, results, attendance, sections, buildings, admins. Prisma fully manages these (create/alter/drop via migrations).
- **Legacy university tables** `LEGACY` — `isr_*` tables that belong to the existing university ERP (student/faculty master data, course catalog, course-faculty mapping, semester-wise registration). The quiz platform *reads* these constantly and *writes* to a few of them only where the admin panel explicitly manages accounts (create/update/delete faculty & students, activate/deactivate logins, faculty-course mapping). Prisma models them 1:1 for type-safety, but conceptually they are not "our" tables — the columns must match the real university export exactly.



Why no foreign keys between the two sides? Every place the app needs to reference a student or faculty member (e.g. `Quiz.facultyRoll`, `QuizAttempt.studentRoll`, `SectionStudent.studentRoll`) stores the legacy `roll number` as a plain string column — *not* a Prisma relation/foreign key into `isr_stu_data_tbl` or `isr_faculty_tbl`. This is deliberate: the legacy tables are outside Prisma's migration control, so a real foreign-key constraint would break the moment the university re-imports or restructures its own data. Every join between "our" tables and "their" tables therefore happens in application code (in `src/lib/legacy-db.ts`), not at the database constraint level.

A third, small layer sits between the two: **legacy-integration config** `CONFIG` — app-owned tables that don't model quiz data at all, but instead configure *how* the app talks to the legacy side (which semester cycle is "current", which batch maps to which physical registration table).

> **Physical location:** every table below lives in the same MySQL schema pointed to by `DATABASE_URL`. Prisma's `@@map(...)` directive is what fixes each model to its real snake_case table name (e.g. model `Quiz` → table `quizzes`; model `IsrStuDataTbl` → table `isr_stu_data_tbl`).

2. Enums Reference

MySQL enum columns used to constrain specific fields to a fixed set of values. Referenced throughout the tables below.

Enum	Values	Used on	
`AdminRole`	`super_admin` `admin`	`admins.role`	
`PersonStatus`	`active` `inactive`	reserved / conceptual status	
`QuizStatus`	`draft` `scheduled` `live` `completed`	`quizzes.status`	
`QuestionType`	`mcq` `formula` `subjective`	`questions.question_type`	
`AllotmentStatus`	`allotted` `attempted` `absent`	`quiz_allotments.status`	
`AttemptStatus`	`in_progress` `submitted` `auto_submitted`	`quiz_attempts.status`	
`AttendanceStatus`	`present` `absent`	`attendance.status`	
`ResultStatus`	`pending` `declared` `published`	`results.status`	
`AntiCheatEventType`	`screen_switch` `overlay_detected` `background`	`anti_cheat_events.event_type`	
`SectionMemberSource`	`auto` `manual_added` `manual_removed`	`section_students.source`	`section_faculty.source`

3. Identity & Master Data

Core reference tables the rest of the app hangs off. Fully app-owned.

`admins` APP

Model: `Admin` — the only login table that belongs entirely to this app (faculty/student logins live in the legacy `isr_login_tbl` instead, see §5).

Column	Type	Notes
`id`	Int, PK	Auto-increment
`name`	String	
`email`	String	Unique — login identifier
`password_hash`	String	bcrypt hash
`role`	AdminRole	Default `admin` ; `super_admin` for elevated access
`refresh_token_hash`	String?	Hashed refresh token for session renewal
`created_at / updated_at`	DateTime	Standard audit timestamps

`departments` APP

Model: `Department` — app-side department catalog (distinct from the legacy "branch/major" codes used inside `isr_*` tables). Created on demand — e.g. when a course is first linked to a section, a department is upserted by name if it doesn't exist yet.

Column	Type	Notes
`id`	Int, PK	
`name`	String	Unique
`created_at / updated_at`	DateTime	

Relations: one department → many `courses` .

`courses` APP

Model: `Course` — the app's own course catalog. Not the same thing as the legacy curriculum table (`isr_curriculum_tbl` , §5) — a course only appears here once an admin (or an auto-linking flow) has actually used it in a quiz/section context. `code` is the join key back to legacy course codes (`isr_sub_available_tbl.sub_code` , `isr_curriculum_tbl.bsms_code`).

Column	Type	Notes
`id`	Int, PK	
`name`	String	Course title (from curriculum, or the bare code as fallback)
`code`	String	Unique — matches legacy `sub_code` / `bsms_code`
`department_id`	Int, FK → departments.id	

`credits`	Int	Default 0
`created_at / updated_at`	DateTime	

Relations: `sectionCourses` (→ section_courses), `quizzes`, `attendance`.

`academic_sessions` APP

Model: `AcademicSession` — a named term (e.g. "Odd 2026") a quiz can optionally be tagged with.

Column	Type	Notes
`id`	Int, PK	
`name`	String	
`start_date / end_date`	DateTime	
`sub_list_code`	String?	Snapshot of `SemesterConfig.currentSubList` at creation time — ties the session to the legacy semester cycle it belongs to
`created_at / updated_at`	DateTime	

`buildings` APP

Model: `Building` — physical exam-center locations used for geofenced quizzes. Each quiz picks one building; the student's live GPS position is checked against it before/during an attempt (see `geofence_logs` in §7).

Column	Type	Notes
`id`	Int, PK	
`name`	String	
`latitude / longitude`	Decimal(10,7)	Center point of the allowed radius
`radius_meters`	Int	How far from the center a student may attempt from
`created_at / updated_at`	DateTime	

4. Section System

"Sections" are the app's own concept — the legacy university database has no notion of a section at all (confirmed against the real dump). A section groups students and faculty around one or more courses, and quizzes are assigned to sections (a quiz can target more than one merged section at once).

`sections` APP

Column	Type	Notes
`id`	Int, PK	
`name`	String	Convention: `Major_Semester`, e.g. `CE_3` — derived from real student data, never freely typed
`created_at / updated_at`	DateTime	

`section_courses` JOIN

Model: `SectionCourse` — many-to-many between sections and courses, so one merged section can span multiple courses/branches (e.g. a combined CS + IT + Mechanical section).

Column	Type	Notes
`id`	Int, PK	
`section_id`	Int, FK → sections.id	Cascade delete
`course_id`	Int, FK → courses.id	Cascade delete
`created_at`	DateTime	

Unique on `(section_id, course_id)`.

`section_students` & `section_faculty` APP

Models: `SectionStudent`, `SectionFaculty` — membership rows keyed by the *legacy* roll number (`student_roll` / `faculty_roll` — plain strings, not Prisma FKs into the isr_* tables, per §1). Every row also carries a `source` that records where it came from:

Column	Type	Notes
`id`	Int, PK	
`section_id`	Int, FK → sections.id	Cascade delete
`student_roll / faculty_roll`	String	Legacy roll — no DB-level FK
`source`	SectionMemberSource	`auto` / `manual_added` / `manual_removed`
`created_at`	DateTime	

Unique on `(section_id, student_roll)` resp. `(section_id, faculty_roll)`.

How membership stays in sync (src/lib/section-sync.ts)

Every section's roster is periodically reconciled against the legacy course rosters of its linked courses (`syncSectionStudents` / `syncSectionFaculty`):

- `auto` rows are pure system state — freely added when a legacy roster gains a matching registrant, freely removed when it no longer does.
- `manual_added` rows are kept even if the legacy roster later drops that person — an explicit admin override.
- `manual_removed` rows are kept (never resurrected) even if the roster still lists that roll — records a deliberate exclusion.

This makes re-sync idempotent and override-preserving: running it repeatedly never undoes a human decision. Student desired-rolls come from `getCourseRegistrations()` against the current semester's `isr_reg_<batch>_tbl` rows; faculty desired-rolls come from `isr_sub_available_tbl` rows whose resolved major matches the section's own major.

> **Denormalized mirror:** every write to `section_students` also updates `isr_stu_main_tbl.section` — a comma-joined list of section names — via `refreshStudentSectionColumn()` . This column doesn't exist in the real university export; it's an app-added convenience column so legacy-facing screens can show section membership without a join, and it gets rebuilt automatically after every legacy data re-import.

5. Legacy University Tables

These `isr_*` tables mirror the real university ERP schema column-for-column. The app treats them as authoritative for identity, academic status, and course offerings. All reads/writes to this layer are centralized in `src/lib/legacy-db.ts` — no other file in the codebase talks to these tables directly.

`isr_login_tbl` LEGACY

Model: `IsrLoginTbl` — PK `user_roll`. Single login table shared by *both* students and faculty, distinguished by `user_type` (`FAC` / `STU`). Users can sign in with roll, email, *or* mobile — login lookup matches whichever identifier was typed.

Column	Type	Notes
`user_roll`	String, PK	
`user_email`	String	
`user_password`	String	bcrypt-compared; legacy hash scheme unconfirmed (flagged TODO in code)
`user_type`	String	`FAC` or `STU`
`status`	Int	1 = Active, 2 = Inactive — flipped by the admin's activate/deactivate action
`user_mobile`	String?	Alternate login identifier
`branch, batch, program_name, centre_location, ...`	various	Full structural parity columns, mostly informational

`isr_faculty_tbl` LEGACY

Model: `IsrFacultyTbl` — PK `roll`. Faculty profile data.

Column	Type	Notes
`roll`	String, PK	
`name`	String	
`dept`	String?	Displayed in admin profile view
`emp_code`	String?	
`fac_status`	String?	R-Regular, V-Visiting, P-PFD, T-Temporary
`~15 more columns`	various	DOB, joining/exit dates, invigilator status, pay level, address, etc. — kept for structural parity, not all surfaced in the UI

`isr_stu_data_tbl` LEGACY

Model: `IsrStuDataTbl` — PK `stu_roll`. A large (~60-column) personal/family/hostel profile table. Only `stu_roll`, `stu_name`, `mobile` are surfaced on list views by default — sensitive fields (Aadhaar, bank details, parent contact info) are pulled in only on a deliberate detail-view request.

Column	Type	Notes
`stu_roll`	String, PK	
`stu_name`	String	
`mobile`	String?	
`email, blood_group, father_/mother_*, bank_*, hostel_*, aadhaar_card, ...`	various	Full personal profile — read-only, sensitive fields not shown by default

`isr_stu_main_tbl` LEGACY

Model: `IsrStuMainTbl` — PK `roll`. Academic-status table: major, batch, current semester, registration/attendance/fee status.

Column	Type	Notes
`roll`	String, PK	
`major`	String	Drives section-name derivation (`Major_Semester`)
`name`	String	
`batch`	String	Resolves to a registration table via `batch_table_registry`
`sem_now`	Int	Real column type is INT; app boundary converts to/from string
`reg_status / attn_ok / stu_status`	String?	Academic-status flags, read-only
`fee_OK, cgpa_OK, hostel_status, present_status, ...`	various	Structural parity columns
`section`	String? (Text)	**App-added** — not part of the real university dump. Comma-joined list of every `sections.name` this student currently belongs to, kept in sync by `refreshStudentSectionColumn()` and rewritten after every legacy re-import (which truncates it, since the raw dump has no such column)

`isr_sub_available_tbl` LEGACY

Model: `IsrSubAvailableTbl` — PK `avai_sr` (mapped to Prisma field `id`). The faculty ⇄ course mapping table: which faculty teaches which subject code, for which branch/semester, under which semester cycle (`sub_list`).

Column	Type	Notes
`avai_sr (id)`	Int, PK	
`sem`	String?	
`sub_list`	String?	Semester-cycle code, e.g. `C2` — admin-configurable "current" value, default `c1`

`sub_code`	String?	Course code — joins to `courses.code` / `isr_curriculum_tbl.bsms_code`
`fac_roll`	String?	Nullable — a mapping row can exist with no faculty assigned yet
`branch`	String?	**Not** the same code space as `isr_stu_main_tbl.major` — some values are prefixed with "D" (e.g. `DCE` for major `CE`); resolved via `resolveMajorFromBranch()`
`b1_sem, sem_list, slot, fac_order`	various	Structural parity columns
`section`	String?	**App-added** — the single `Major_Semester` section this mapping row resolves to, set once at creation and rewritten after every legacy re-import

`isr_curriculum_tbl` LEGACY

Model: `IsrCurriculumTbl` — PK `sr` (mapped to Prisma field `id`). The full curriculum/course catalog — resolves a bare `sub_code` (from `isr_sub_available_tbl` or a registration table) into a human-readable title, credits and branch.

Column	Type	Notes
`sr (id)`	Int, PK	
`sub_list`	String	Semester cycle
`bsms_branch`	String	
`bsms_code`	String	Course code — indexed
`title`	String (Text)	Human-readable course title
`l / t / p`	Int	Lecture/tutorial/practical hours
`bsms_credit`	Int	
`sem`	String?	
`sp, dept_core, dept_elec, elec_num, open_elec, dept_minor`	various	Structural parity columns

`isr_reg_<batch>_tbl` *(dynamic)* LEGACY

Not a Prisma model — one physical table *per student batch* (e.g. `isr_reg_btechpeg23_tbl`), holding that batch's actual course registrations for the current semester cycle. Accessed exclusively via raw parameterized SQL in `legacy-db.ts`, gated by an allow-list (`batch_table_registry`, §6) so a table name is never built from unvalidated input.

Column	Notes
`stu_roll`	Student roll (note: *not* `roll` — this table alone uses `stu_roll`)
`sub_code`	Registered course code
`sub_list`	Semester cycle this registration belongs to
`reg_sr`	Real auto-increment PK — used to order "most recent first"

> **Coverage gap (by design, documented in code):** only 3 of the real student batches (`btechpeg23/24/25`) actually have a registration table — roughly 94% of students, by roll count, have none. Filtering logic therefore only *excludes* a student from a course roster when their own batch has a real registration table saying they aren't registered; students whose batch has no table at all are left unfiltered, since section membership is the best available signal for them.

6. Legacy Integration Config

Small app-owned tables that don't represent quiz or student data — they configure how the app talks to the legacy layer above.

`semester_config` CONFIG

Model: `SemesterConfig` — a single-row table holding the "current" semester cycle code, editable by an admin. Every legacy read that needs a `sub_list` value (course rosters, faculty mappings, curriculum lookups) defaults to this value unless a specific one is passed.

Column	Type	Notes
`id`	Int, PK	
`current_sub_list`	String	e.g. `C2`
`updated_at`	DateTime	

`batch_table_registry` CONFIG

Model: `BatchTableRegistry` — allow-list mapping a batch name to its physical `isr_reg_<batch>_tbl` table name. Exists purely so dynamic-table SQL queries never build a table identifier from unvalidated input (SQL-injection guard).

Column	Type	Notes
`id`	Int, PK	
`batch_name`	String	Unique, e.g. `btechpeg23`
`table_name`	String	Real physical table, e.g. `isr_reg_btechpeg23_tbl`
`is_active`	Boolean	Default true — inactive entries are skipped by every registration query
`created_at`	DateTime	

7. Quiz Domain

The core product: everything about creating, running, answering, and grading a quiz. All fully app-owned.

`quizzes` APP

Column	Type	Notes
`id`	Int, PK	
`title`	String	
`course_id`	Int, FK → courses.id	
`session_id`	Int?, FK → academic_sessions.id	Optional
`faculty_roll`	String	Legacy roll — no DB-level FK, indexed
`building_id`	Int, FK → buildings.id	For geofencing
`start_time / end_time`	DateTime	Scheduled window
`duration_minutes`	Int	
`total_marks`	Int	
`randomize`	Boolean	Default true — shuffle question order per student
`negative_marking`	Boolean	Default false
`allow_skip_switch`	Boolean	Default true
`require_location`	Boolean	Default true — gates geofence checks
`status`	QuizStatus	draft → scheduled → live → completed
`actual_start_time / actual_stop_time`	DateTime?	Set when faculty actually starts/stops (vs. the scheduled window)
`deleted_at`	DateTime?	Soft delete
`created_at / updated_at`	DateTime	

Indexes: `course_id`, `session_id`, `faculty_roll`, `building_id`, `status`. Relations: `sections` (via quiz_sections), `questions`, `allotments`, `attempts`, `attendance`, `results` — all cascade-deleted with the quiz.

`quiz_sections` JOIN

Model: `QuizSection` — many-to-many so one quiz can target more than one merged section at once (section selection is mandatory at quiz-creation time).

Column	Type	Notes
`id`	Int, PK	
`quiz_id`	Int, FK → quizzes.id	Cascade delete

`section_id`	Int, FK → sections.id	Cascade delete
`created_at`	DateTime	

Unique on `(quiz_id, section_id)`.

`questions`, `question_options`, `question_formula` APP

One quiz has many questions; a question's shape depends on its `question_type`:

- `mcq` → has rows in `question_options`, one flagged `is_correct`
- `formula` → has exactly one row in `question_formula` (numeric answer + tolerance)
- `subjective` → free-text, graded manually; may carry an optional `reference_answer` for the grader (never sent to students)

Table	Column	Type	Notes
`questions`	`id`	Int, PK	
	`quiz_id`	Int, FK → quizzes.id	Cascade delete, indexed
	`question_text`	Text	
	`question_type`	QuestionType	mcq / formula / subjective
	`marks / negative_marks`	Int	
	`reference_answer`	Text?	Subjective-only grading guide
	`order_index`	Int	Display order
`question_options`	`id / question_id (FK, cascade)`	Int	Indexed on question_id
	`option_text`	Text	
	`is_correct`	Boolean	Default false
`question_formula`	`id / question_id (FK, cascade, unique)`	Int	1:1 with question
	`correct_value`	Float	
	`tolerance`	Float	Acceptable margin for a numeric match

`quiz_allotments` APP

Model: `QuizAllotment` — which students are eligible to attempt a given quiz, and their current status.

Column	Type	Notes
`id`	Int, PK	
`quiz_id`	Int, FK → quizzes.id	Cascade delete

`student_roll`	String	Legacy roll, indexed
`status`	AllotmentStatus	allotted / attempted / absent
`is_proxy`	Boolean	Default false — flags a live attempt suspected to be a proxy (someone else attempting on the student's behalf). Always forces status to `absent`; un-flagging restores real attendance
`created_at / updated_at`	DateTime	

Unique on `(quiz_id, student_roll)`.

`quiz_attempts` APP

Model: `QuizAttempt` — one row per student per quiz once they actually start.

Column	Type	Notes
`id`	Int, PK	
`quiz_id`	Int, FK → quizzes.id	Cascade delete
`student_roll`	String	Legacy roll, indexed
`start_time / end_time`	DateTime	
`status`	AttemptStatus	in_progress / submitted / auto_submitted
`latitude / longitude`	Decimal(10,7)?	Captured at attempt start for geofencing
`auto_submitted`	Boolean	True if the system force-submitted (time up, violation limit, etc.)
`auto_submit_reason`	String?	
`question_order`	Text?	Serialized per-student randomized question order (so a submitted attempt can always be re-rendered in the order the student actually saw)
`created_at / updated_at`	DateTime	

Unique on `(quiz_id, student_roll)` — one attempt per student per quiz. Relations: `answers`, `antiCheatEvents`, `geofenceLogs` — all cascade-deleted with the attempt.

`student_answers` APP

Model: `StudentAnswer` — one row per question per attempt, shape depends on the question type.

Column	Type	Notes
`id`	Int, PK	
`attempt_id`	Int, FK → quiz_attempts.id	Cascade delete
`question_id`	Int, FK → questions.id	Cascade delete, indexed

selected_option_id	Int?, FK → question_options.id	MCQ answers, indexed
answer_value	Float?	Formula-type numeric answer
written_answer	Text?	Subjective free-text answer
manually_graded	Boolean	Default false — flips true once faculty scores a subjective answer
is_correct	Boolean?	Auto-evaluated for MCQ/formula
marks_obtained	Float	Default 0 until graded
order_index	Int	
is_skipped	Boolean	Default false
created_at / updated_at	DateTime	

Unique on `(attempt_id, question_id)` — a student answers each question at most once.

attendance APP

Column	Type	Notes
id	Int, PK	
student_roll	String	Legacy roll, indexed
course_id	Int, FK → courses.id	Cascade delete, indexed
quiz_id	Int, FK → quizzes.id	Cascade delete
date	Date	Indexed
status	AttendanceStatus	present / absent, indexed
created_at / updated_at	DateTime	

Unique on `(student_roll, quiz_id)` — attendance is derived from quiz participation, one record per student per quiz.

results APP

Column	Type	Notes
id	Int, PK	
quiz_id	Int, FK → quizzes.id	Cascade delete
student_roll	String	Legacy roll, indexed
marks_obtained	Float	Default 0
percentage	Float	Default 0
status	ResultStatus	pending → declared → published, indexed

`declared_at / published_at`	DateTime?	
`created_at / updated_at`	DateTime	

Unique on `(quiz_id, student_roll)`.

`anti_cheat_events` APP

Model: `AntiCheatEvent` — logs suspicious behavior during a live attempt (tab switches, overlay apps detected, app backgrounded).

Column	Type	Notes
`id`	Int, PK	
`attempt_id`	Int, FK → quiz_attempts.id	Cascade delete, indexed
`event_type`	AntiCheatEventType	screen_switch / overlay_detected / background
`event_time`	DateTime	Default now()
`action_taken`	String?	e.g. "warned", "auto-submitted"

`geofence_logs` APP

Model: `GeofenceLog` — every location check performed during an attempt, against the quiz's assigned building.

Column	Type	Notes
`id`	Int, PK	
`attempt_id`	Int?, FK → quiz_attempts.id	Cascade delete, indexed, nullable (a check can happen before an attempt row exists)
`quiz_id`	Int	Indexed
`student_roll`	String	Legacy roll, indexed
`latitude / longitude`	Decimal(10,7)	Student's reported position
`distance_meters`	Float	Distance from the building's center point
`is_within_range`	Boolean	Result of comparing distance to `buildings.radius_meters`
`checked_at`	DateTime	Default now()

8. End-to-End Workflows

8.1 Login

User enters roll / email / mobile → isr_login_tbl (match on any identifier, scoped by user_type) → bcrypt.compare() (against user_password) → JWT session (roll carried as the identity)

Admins are the one exception — they authenticate purely against the app-owned `admins` table, never touching `isr_login_tbl`.

8.2 Section membership sync

Section linked to Course(s) (via section_courses) → syncSection() (reads isr_reg_<batch>_tbl + isr_sub_available_tbl) → section_students / section_faculty reconciled → isr_stu_main_tbl.section rewritten (mirror column)

8.3 Quiz lifecycle

Faculty creates Quiz (picks course, section(s), building) → Questions added (mcq / formula / subjective) → Students allotted (quiz_allotments) → Faculty starts quiz (status → live)

Student starts attempt (quiz_attempts row created, GPS captured) → Geofence checked (geofence_logs) → Answers saved live (student_answers, upserted per question) → Submit / auto-submit (attempt.status updated)

Faculty grades subjective answers (marks_obtained, manually_graded=true) → Result computed (results row, status=declared) → Faculty publishes (status=published, published_at set) → Student sees result

Attendance (`attendance` table) is derived alongside the attempt/allotment lifecycle: an allotted student who never attempts is `absent`; one who does is `present`, unless later flagged `is_proxy` on their allotment, which forces them back to absent.

8.4 Anti-cheat during a live attempt

Client detects violation (tab switch / overlay / backgrounded) → anti_cheat_events row logged → Policy check (allow_skip_switch, violation count) →

Optional auto-submit
attempt.auto_submitted=true

8.5 Faculty ⇄ Course ⇄ Section mapping (legacy-grounded)

The Create/Edit Quiz section picker deliberately does *not* read the app's own `section_faculty` / `section_students` tables — per an explicit product decision, quiz creation's course → section → student chain must trace back to the real `isr_*` tables live, on every call:

isr_sub_available_tbl
facRoll + subCode + subList → section name

→

isr_reg_<batch>_tbl
confirms real registration

→

isr_stu_main_tbl
resolves major → roll

→

Candidate student list
for allotment

Master Data > Sections and Manage Members still use the app's own `Section` tables — only the quiz-creation candidate list was changed to read live from the legacy tables.

9. Key Design Decisions & Gotchas

- **No foreign keys across the app/legacy boundary.** Every `*_roll` column (student_roll, faculty_roll) is a bare string, joined in application code. Protects the app from breaking when the university re-imports or restructures its own tables.
- **Legacy write surface is narrow and explicit.** The app only writes to `isr_login_tbl`, `isr_faculty_tbl`, `isr_stu_data_tbl`, `isr_stu_main_tbl`, and `isr_sub_available_tbl` — and only through admin-panel CRUD (create/update/delete faculty & students, faculty-course mapping, login activation). Everything else (`isr_curriculum_tbl`, `isr_reg_<batch>_tbl`) is strictly read-only from this app.
- **Semester cycles (`sub_list`).** Nearly every legacy read is scoped to a "current" semester-cycle code (`c1`, `C2`, ...) resolved from `semester_config`. Academic sessions snapshot this value at creation time so historical quizzes stay tied to the cycle they were actually run under.
- **Dynamic table access is allow-listed.** `batch_table_registry` exists solely to prevent building a raw SQL table name from unvalidated input — every `isr_reg_<batch>_tbl` query first resolves the batch through this registry and validates the name against a strict pattern.
- **App-added columns inside legacy tables.** `isr_stu_main_tbl.section` and `isr_sub_available_tbl.section` are not part of the real university export — they're denormalized conveniences the app maintains and rebuilds after every legacy re-import.
- **`branch` vs. `major` mismatch.** `isr_sub_available_tbl.branch` uses a different code space than `isr_stu_main_tbl.major` (e.g. `DCE` vs. `CE`) — always resolved through `resolveMajorFromBranch()`, never compared directly.
- **Soft delete on quizzes.** `quizzes.deleted_at` means a quiz is hidden, not physically removed — its questions, attempts, results, and attendance history stay intact.
- **Proxy flag forces absence.** `quiz_allotments.is_proxy` always overrides `status` to `absent` while set, regardless of whether an attempt exists — un-flagging restores the student's real attendance.
- **Manual overrides always win in section sync.** `manual_added` / `manual_removed` rows are never touched by the auto-sync reconciliation loop — this is what makes re-running sync safe at any time.

10. Migration History

Schema evolution, in order, as tracked under `prisma/migrations/`:

Migration	What it did
`20260805044028_init`	Initial schema — core app tables + first pass at the isr_* mirror tables
`20260807120000_legacy_curriculum_and_profile_fields`	Added `isr_curriculum_tbl` and extra profile columns
`20260807130000_subjective_questions`	Added subjective question type support (`reference_answer`, `written_answer`, `manually_graded`)
`20260808120000_section_hub_redesign`	Introduced the Section system (sections, section_courses, section_students, section_faculty) replacing an earlier model
`20260808140000_quiz_allotment_proxy_flag`	Added `quiz_allotments.is_proxy`
`20260808150000_quiz_location_and_soft_delete_fields`	Added `quizzes.deleted_at` and location/geofencing fields
`20260809100000_academic_session_sub_list_code`	Added `academic_sessions.sub_list_code` snapshot
`20260810080000_legacy_full_schema_parity`	Brought `isr_faculty_tbl` / `isr_stu_data_tbl` / `isr_stu_main_tbl` / `isr_sub_available_tbl` up to full column parity with the real university export
`20260810090000_legacy_reg_table_full_parity`	Confirmed/aligned `isr_reg_<batch>_tbl` real columns (`reg_sr` PK, `stu_roll`, `sub_code`, `sub_list`)
`20260810100000_sub_available_nullable_fields`	Corrected nullability on `isr_sub_available_tbl` columns to match the real table
`20260811150000_add_section_columns_to_isr_tables`	Added the app-owned `section` mirror columns to `isr_stu_main_tbl` and `isr_sub_available_tbl`

Migration provider: MySQL, applied via `prisma migrate`. Client generated with `prisma-client-js`.